

1. Considere um programa que rastreie os itens na biblioteca de uma Universidade. Desenhe a hierarquia de classes, incluindo uma classe base, para os diferentes tipos de itens. Certifique-se de considerar também os itens que não podem ser requisitados.
2. Desenhe uma hierarquia para os componentes que você pode encontrar numa interface gráfica do utilizador. Observe que alguns componentes podem acionar ações em resposta a eventos. Alguns componentes podem ter gráficos associados a eles. Alguns componentes podem conter outros componentes.
3. Suponha que queremos implementar um programa de desenho que crie várias formas usando caracteres do teclado. Implemente uma classe base abstrata `DrawableShape` que conheça o centro (dois valores inteiros) e a cor (uma string) do objeto. Forneça métodos de acesso apropriados para os atributos. Você também deve ter um método modificador que mova o objeto por uma determinada quantidade.
4. Crie uma classe `Square` derivada de `DrawableShape`, conforme descrito no exercício anterior. Um objeto `Square` deve saber o comprimento de seus lados. A classe deve ter um método de acesso e um método de mutação para este comprimento. Deve também ter métodos para calcular a área e o perímetro do quadrado.
5. Crie uma classe abstrata `PayCalculator` que tenha um atributo `payRate` dado em dólares por hora. A classe também deve ter um método `computePay(hours)` que retorne o pagamento por um determinado período de tempo.
6. Derive uma classe `RegularPay` de `PayCalculator`, conforme descrito no exercício anterior. Deve ter um construtor que tenha um parâmetro para a taxa de pagamento. Ele não deve substituir nenhum dos métodos. Em seguida, derive uma classe `HazardPay` de `PayCalculator` que substitui o método `computePay`. O novo método deve retornar o valor retornado pelo método da classe base multiplicado por 1.5.
7. Crie uma classe abstrata `DiscountPolicy`. Deve ter um único método abstrato `computeDiscount` que retornará o desconto para a compra de um determinado número de um único item. O método tem dois parâmetros, `count` e `itemCost`.
8. Derive uma classe `BulkDiscount` de `DiscountPolicy`, conforme descrito no exercício anterior. Deve ter um construtor que tenha dois parâmetros, `mínimo` e `percentual`. Deve definir o método `computeDiscount` para que se a quantidade comprada de um item for maior que o mínimo, o desconto seja `percentual`.

9. Derive uma classe `BuyNItemsGetOneFree` de `DiscountPolicy`. A classe deve ter um construtor que tenha um único parâmetro `n`. Além disso, a classe deve definir o método `computeDiscount` para que todo enésimo item seja gratuito. Por exemplo, a tabela a seguir fornece o desconto para a compra de várias contagens de um item que custa \$ 10, quando `n` é 3

Count	1	2	3	4	5	6	7
Discount	0	0	10	10	10	20	20

10. Derive uma classe `CombinedDiscount` de `DiscountPolicy`. Deve ter um construtor que tenha dois parâmetros do tipo `DiscountPolicy`. Ele deve definir o método `computeDiscount` para retornar o valor máximo retornado por `computeDiscount` para cada uma de suas duas políticas de desconto privadas. As duas políticas de desconto estão descritas nos dois exercícios anteriores.
11. Implemente uma superclasse `Person`. Implementa também duas classes, `Student` e `Instructor`, que herdam da `Pessoa`. Uma pessoa tem um nome e um ano de nascimento. Um aluno tem um diploma e um instrutor tem um salário. Escreva as declarações de classe, os construtores e o métodos `toString` para todas as classes. Forneça um programa de teste que teste essas classes e métodos.
12. Declara a seguinte interface:

```
public interface Filter
{
    boolean accept(Object x);
}
```

Esta interface tem o método que recebe um objecto como parâmetro e retorna booleano.

- Implementa uma classe `ShortWordFilter` que implementa a interface `Filter` e cujo método `accept` retorna `true` para todas as strings de comprimento menor que um valor tamanho especificado como atributo da classe.
- Implementa uma classe de teste que leia 5 palavras de `System.in`, coloque-as em um `ArrayList<String>`, e de seguida chama o método `collectAll` que retorna uma lista das palavras de dimensão menor que cinco. [Para isso deverá usar o método implementado na classe do ponto anterior]
- O método que é referido no ponto anterior deve ter a seguinte assinatura `public static ArrayList<Object> collectAll(ArrayList<Object> objects, Filter f)` e deve retornar todos os objectos no array que são aceites (portanto que o método `accept` retorne `true`) pelo filtro fornecido.

